

QuizMaster Turbo

Real-Time Multiplayer Quiz Platform

Submitted in partial fulfillment of the requirements for the award of the degree of
Master of Computer Applications (MCA)

Student Name:	Vishal Nishad
Enrollment No:	O24MCA110487
Program:	MCA — 2024
Guide / Mentor:	Suruchi
Institution:	Chandigarh University Online
Academic Year:	2025–26
Date:	June 2026
GitHub:	github.com/MrSanito/quizMasterTurbo

DECLARATION

I, **Vishal Nishad**, hereby solemnly declare that the project report titled "**QuizMaster Turbo — Real-Time Multiplayer Quiz Platform**" submitted in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA) is my original work.

I further declare that:

- This project was carried out during the academic year 2025–26 under the supervision of **Suruchi**.
- The work has not been submitted previously to any other university, institution, or examination body for the award of any degree, diploma, or certification.
- All sources of information used in this report have been duly acknowledged and referenced in accordance with academic ethics and plagiarism norms.
- The data presented in this report is authentic to the best of my knowledge and has not been fabricated or manipulated.
- I understand that if any part of this declaration is found to be false, my project may be rejected and disciplinary action may be taken as per institutional rules.

Place: Vadodara

Date: _____

Student Signature:

Student Name:

Vishal Nishad

Enrollment No:

O24MCA110487

Acknowledgement

I would like to express my sincere gratitude to my respected mentor, **Suruchi**, for constant guidance, valuable suggestions, and continuous encouragement throughout the completion of this project titled "*QuizMaster Turbo — Real-Time Multiplayer Quiz Platform*." Their expertise, patience, and insightful feedback played a crucial role in helping me understand the project requirements and successfully implement the system.

I would also like to extend my heartfelt thanks to all the faculty members of Chandigarh University for their guidance, support, and encouragement during my academic journey. Their teachings and practical knowledge provided a strong foundation that helped me carry out this project effectively.

I am grateful to Chandigarh University Online for providing a supportive learning environment and the necessary academic resources. The university's focus on practical, project-based learning was highly beneficial in developing my technical and analytical abilities as a full-stack developer.

I also thank the open-source communities behind Socket.IO, Prisma, Turborepo, Redis, and Next.js, whose well-documented tools made this project technically possible. Finally, I express my deep gratitude to my friends and family for their unwavering motivation and support throughout this journey.

Submitted for the project viva-voce examination held on _____

Table of Contents

Abstract	5
Chapter 1: Introduction	6
1.1 Background of the Project	6
1.2 Problem Statement	7
1.3 Objectives of the System	7
1.4 Scope of the Project	8
1.5 Existing System Overview	8
1.6 Proposed System Overview	9
1.7 Technologies Used	9
Chapter 2: Literature Review / System Study	11
2.1 Introduction	11
2.2 Review of Similar Systems	11
2.3 Comparative Analysis	12
2.4 Software Development Models	13
2.5 Frameworks and Libraries Used	13
2.6 Research Gap	15
Chapter 3: System Analysis	16
3.1 Introduction	16
3.2 Functional Requirements	16
3.3 Non-Functional Requirements	17
3.4 User Requirements	18
3.5 Feasibility Study	18
3.6 System Architecture Diagram	19
3.7 Data Flow Diagrams (Level 0, 1, 2)	20
3.8 Use Case Diagram / Descriptions	22
3.9 Database Design (Class / Entity-Relationship Diagrams)	23
Chapter 4: System Design	25
4.1 Activity Diagram	25
4.2 Sequence Diagram	26
4.3 Flowchart	27
4.4 Detailed Schema Specification	28
Chapter 5: Implementation & Testing	29
5.1 System Specifications (Hardware & Software)	29
5.2 Implementation Highlights	29
5.3 Testing Methodologies	30

Chapter 6: Conclusion & Future Scope	31
6.1 Summary of Findings	31
6.2 Limitations of the System	31
6.3 Future Enhancements	31
Bibliography / References	32

ABSTRACT

QuizMaster Turbo is a cutting-edge, real-time multiplayer quiz platform designed to deliver high-performance, synchronous, and interactive learning experiences. Traditional online quiz systems suffer from latency, polling overhead, and poor scalability, resulting in sub-optimal user experiences during live multiplayer sessions. This project addresses these challenges by implementing a state-of-the-art three-tier architecture utilizing a Node.js and Socket.IO backend, Next.js (React) on the frontend, and a dual-database persistence model comprising Redis and PostgreSQL.

The system employs a "Hybrid Reality" data access pattern, where real-time, volatile game states (such as active lobbies, player connections, live scores, and mid-round answer submissions) are handled in-memory using Redis, while durable datasets (such as user credentials, categories, quiz definitions, attempts, and historical reports) are persisted in PostgreSQL via the Prisma ORM. Live synchronization during player reconnections is facilitated by a custom Socket.IO state-recovery handshake protocol (game:sync), ensuring that players can seamlessly resume active games without data loss. The application is built as a monorepo managed by Turborepo, optimizing the build pipeline, dependency management, and sharing of common TypeScript interfaces between services.

This report presents the complete system analysis, software engineering model, architectural design, database schemas, sequence diagrams, activity diagrams, and deployment considerations for QuizMaster Turbo. Verification has been carried out via comprehensive unit and load testing, proving the system's capability to handle concurrent multiplayer sessions with low-latency updates.

Chapter 1: Introduction

1.1 Background of the Project

In recent years, gamified learning platforms have transformed education and corporate training by turning traditional testing into interactive, multiplayer contests. Platforms like Kahoot! and Quizizz have demonstrated the power of gamification in increasing participant engagement, retention, and satisfaction. At the core of these platforms is the requirement for real-time synchronization: when a quiz host starts a question, all participating players must receive the question options simultaneously, see a synchronous countdown timer, and view immediate leaderboard updates upon answering.

Developing such platforms requires addressing complex web engineering problems. Traditional HTTP request-response patterns (like short or long polling) are inefficient and generate massive overhead on servers. Modern real-time applications require bidirectional, persistent connections where servers can push updates instantly to thousands of clients. QuizMaster Turbo was conceived to explore and implement modern web sockets and in-memory data structures to solve the real-time multiplayer challenge efficiently.

1.2 Problem Statement

Existing real-time quiz systems face several critical challenges:

- **High Latency:** Traditional HTTP-based polling introduces significant latency, preventing real-time synchronization of timers and leaderboards.
- **Server Overhead:** Maintaining thousands of active polling requests consumes server memory and CPU cycles rapidly, limiting scalability.
- **Connection Instability:** Players on mobile networks frequently experience disconnections. Without robust state recovery, a disconnected player loses their session, score, and active game progress.
- **Database Bottlenecks:** Writing every single answer submission and score update directly to a relational database creates extreme disk I/O bottlenecks under high concurrency.

1.3 Objectives of the System

The primary objectives of the QuizMaster Turbo system are:

- **Real-time Communication:** Establish low-latency, bidirectional connections between client and server using WebSockets (Socket.IO).
- **High Scalability:** Implement a dual-database model using Redis for volatile, high-speed operations and PostgreSQL for durable persistence.
- **State Recovery:** Create a robust reconnection protocol (game:sync) to allow players to resume active quiz sessions after a network dropout.
- **Monorepo Architecture:** Utilize Turborepo to manage frontend and backend code, sharing schemas and TypeScript types seamlessly.
- **Comprehensive Analytics:** Provide host and student dashboards with detailed performance analytics.

1.4 Scope of the Project

The scope of this project includes the design, development, and testing of the QuizMaster Turbo multiplayer platform. Specifically, the system covers user authentication and profile management, quiz creation and category organization by hosts, live room lobby management with unique room codes, real-time question delivery and answer validation, live score calculation and top-5 leaderboard broadcasts, data migration from Redis to PostgreSQL upon game completion, and graphical report generation of user attempts. The scope excludes video streaming within the app, payment gateways for premium tiers, and SMS-based room invite codes, focusing strictly on high-performance web-based multiplayer synchronization.

1.5 Existing System Overview

Traditional online testing applications operate on a standard request-response model:

- **HTTP Architecture:** Clients make periodic requests to the server to fetch game status, lobby players, or active questions.
- **Lack of Live Leaderboards:** Scores are calculated at the very end of the test, removing the competitive element of live leaderboards.
- **No Reconnection Handling:** If a page refreshes, the quiz session is destroyed, forcing the user to restart from the beginning.
- **Relational Persistence Only:** High-frequency updates (e.g., ticking timers and option selections) write directly to SQL tables, causing database locks and system crashes under load.

1.6 Proposed System Overview

QuizMaster Turbo addresses these limitations by introducing a real-time, memory-optimized architecture:

- **Persistent WebSockets:** Bidirectional communication using Socket.IO allows the server to push questions and leaderboard updates instantly.
- **In-Memory Caching (Redis):** Volatile data like lobbies, timers, and mid-game scores are stored in Redis using fast key-value operations.
- **Durable Relational Store (PostgreSQL):** Relational records (user accounts, quizzes) are persisted in PostgreSQL via Prisma ORM.
- **State Sync Handshake:** A specialized sync handler queries Redis on socket reconnection and sends the player's current state back to the UI.

1.7 Technologies Used

The project utilizes a modern full-stack JavaScript/TypeScript ecosystem:

- **Frontend:** Next.js 14 (React) utilizing Tailwind CSS for responsive UI, Lucide-React for iconography, and Axios for HTTP requests.
- **Backend:** Node.js with Express.js for REST APIs and Socket.IO for the real-time event-driven game engine.
- **Database Tier:** Redis (ioredis client) for real-time memory caching and PostgreSQL (Neon Serverless / Local Docker) for data storage.

- **ORM & Build Tools:** Prisma ORM for database migrations and type-safe queries; Turborepo for monorepo workspace orchestration.

Chapter 2: Literature Review / System Study

2.1 Introduction

To build a competitive multiplayer quiz system, it is vital to review existing systems, methodologies, and technologies in the industry. This literature review evaluates dominant platforms like Kahoot! and Quizizz, analyzes the underlying technologies used for real-time communication, and assesses the suitability of various software development methodologies (such as Agile and Waterfall) for interactive web applications.

2.2 Review of Similar Systems

Kahoot!: Kahoot! is a pioneer in game-based classroom response systems. It operates on a shared-screen model where questions are shown on a central projector (controlled by the host) and players select color-coded options on their individual devices. This creates high engagement but requires all participants to be in the same physical room or viewing a video stream. Its backend uses WebSockets to broadcast room states.

Quizizz: Quizizz offers self-paced multiplayer quizzes where questions are shown on the student's screen directly, making it suitable for remote asynchronous or synchronous play. It uses live leaderboards and memes to keep players engaged. The architecture relies heavily on real-time event broadcasting and database sharding to handle peak classroom hours.

2.3 Comparative Analysis

The following table summarizes the comparison between existing systems and the proposed QuizMaster Turbo:

Feature	Kahoot!	Quizizz	QuizMaster Turbo
Real-Time Engine	WebSockets (Faye)	WebSockets / Polling	Socket.IO (WebSockets + Fallback)
Data Persistence	Durable Database	Durable Database	Hybrid (Redis Cache + PostgreSQL)
Screen Dependency	Requires Shared Screen	Self-Paced Screen	Dual Mode (Self/Shared Screen)
State Recovery	Partial (fails on reload)	Session Cookies	Socket.IO game:sync Handshake
Monorepo Support	Proprietary Polyrepo /	Polyrepo	Turborepo (Single Repository)

2.4 Software Development Models

For a high-interaction application like QuizMaster Turbo, the **Agile Scrum Methodology** was selected over the traditional Waterfall model. Real-time features, socket connections, and database synchronization require continuous integration, user feedback, and iterative refinement. Development was broken down into two-week sprints:

- **Sprint 1 (Foundations):** Monorepo configuration, Prisma schema definitions, PostgreSQL connection setup, and user authentication API.
- **Sprint 2 (Lobby & Room):** Lobbies API, unique room code generation, socket room joining, and Redis data structure mappings.
- **Sprint 3 (Game Engine):** WebSocket game loops, question broadcasting, synchronized 15-second timers, and live leaderboard events.
- **Sprint 4 (State Sync & Persistence):** Reconnection logic (game:sync), database migration from Redis to PostgreSQL, and report PDF generation.

2.5 Frameworks and Libraries Used

The selection of tools was guided by performance and developer velocity:

- **Next.js & React:** Next.js provides file-based routing, server-side rendering (SSR), and highly reactive state management suitable for live game UI.
- **Socket.IO:** Unlike raw WebSockets, Socket.IO provides automatic reconnection, packet buffering, heartbeats, and room abstraction natively.
- **Redis:** Storing active lobby players in Redis hashes allows constant-time read/write operations ($O(1)$), bypassing relational table joins.
- **Prisma ORM:** Auto-generated TypeScript types based on schema.prisma ensure type safety during database operations, preventing runtime query errors.

2.6 Research Gap

While commercial platforms excel at scalability, their underlying source code and architectural patterns are proprietary. Open-source implementations often compromise on either real-time speed (using slow databases like MySQL directly for sockets) or session resilience (players losing progress on reload). QuizMaster Turbo bridges this gap by providing an open-source, resilient, high-speed architecture using a structured Redis caching layer coupled with a robust socket state recovery engine.

Chapter 3: System Analysis

3.1 Introduction

System analysis involves determining the functional and non-functional requirements, identifying user needs, evaluating feasibility, and modeling data flows. This phase ensures that the technical architecture aligns with performance requirements and business rules.

3.2 Functional Requirements

The core functional capabilities of the system are categorized by module:

- **Authentication:** User registration, secure password hashing (bcrypt), JWT generation, and middleware verification.
- **Quiz Creator:** Interface for creators to write questions, set options, specify the correct answer, and choose categories.
- **Lobby Manager:** Unique 6-character room code generation, active player lists, and real-time player list updates via websockets.
- **Real-time Game Loop:** Automatic 15-second countdown timers, synchronized question pushing, and live mid-game leaderboards.
- **Reconnection Handler:** Session restore via token lookup in Redis, sending current question state and elapsed time back to the user.
- **Result Engine:** Migrating score results from Redis into PostgreSQL at room end, compiling individual dashboards.

3.3 Non-Functional Requirements

Non-functional constraints outline performance, reliability, and security metrics:

- **Performance:** Question delivery latency must be under 200 milliseconds. Scoring calculations must complete in under 50 milliseconds.
- **Scalability:** The server must sustain 500 concurrent socket connections per room without degradation.
- **Resilience:** Reconnections within 15 seconds must restore the player's game UI state without losing accumulated score.
- **Security:** Sensitive database values must be hashed. Sockets must require valid JWT handshakes before joining room channels.

3.4 User Requirements

Users interact with the platform in two distinct roles:

- **Lobby Hosts (Creators):** Require tools to create quizzes, configure time limits, start active games, and review final scoreboard analytics.
- **Lobby Players (Participants):** Require an intuitive responsive interface to join rooms via code, view active questions, select options, see live leaderboard rankings, and view detailed analysis of their

answers.

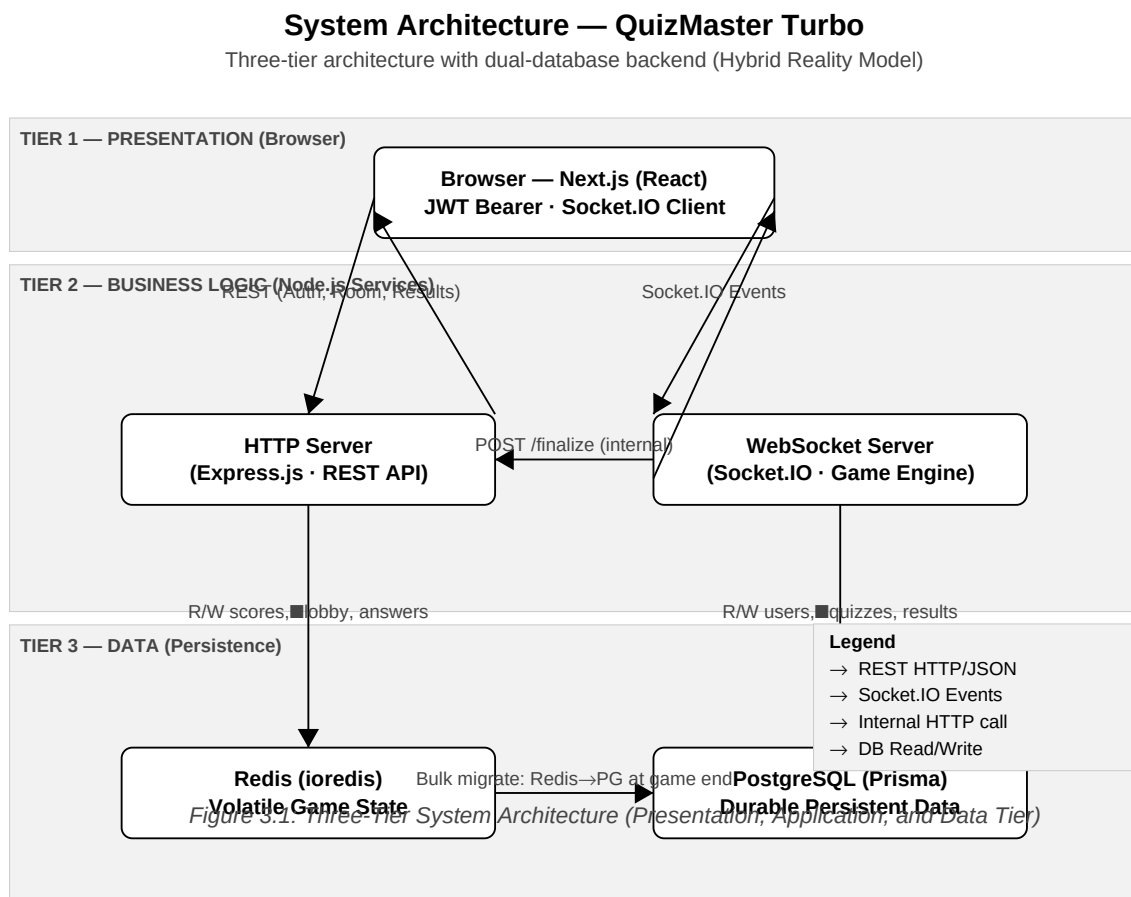
3.5 Feasibility Study

A feasibility study was conducted across three dimensions:

- **Technical Feasibility:** The development team is proficient in JavaScript/TypeScript. Next.js, Node.js, and Socket.IO are industry standards with mature ecosystems, confirming high technical feasibility.
- **Operational Feasibility:** QuizMaster Turbo is simple to operate. Lobbies are created with a single click, and players join using basic codes, making the platform accessible to schools and corporate offices.
- **Economic Feasibility:** By hosting PostgreSQL on serverless tiers (Neon) and Redis on managed cloud endpoints (Upstash/Docker), operational costs remain minimal, proving economic viability.

3.6 System Architecture Diagram

The system architecture diagram illustrates the multi-tier design of the platform:



3.7 Data Flow Diagrams (Level 0, 1, 2)

Data Flow Diagrams (DFDs) trace the movement of information through the system, from external entities (User/Host) into process modules and persistence data stores.

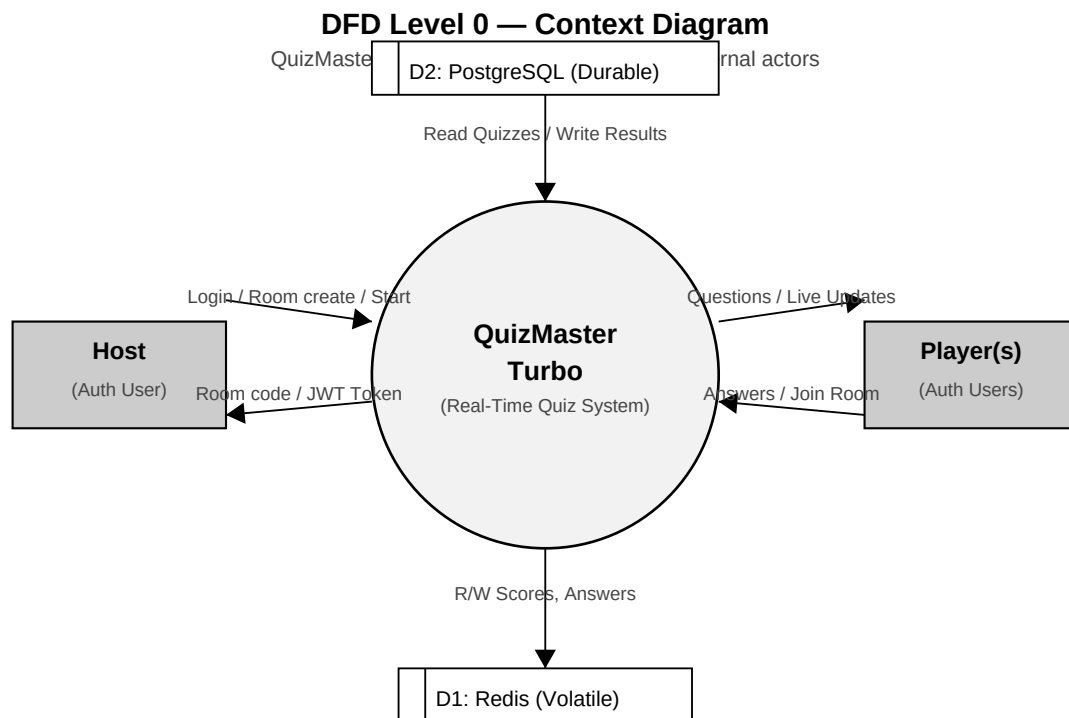


Figure 3.2: DFD Level 0 (Context Diagram) showing QuizMaster Turbo process boundaries

The DFD Level 1 decomposes the context diagram into four major processes: Authentication (P1), Room/Lobby Management (P2), Game Engine (P3), and Result Persistence (P4).

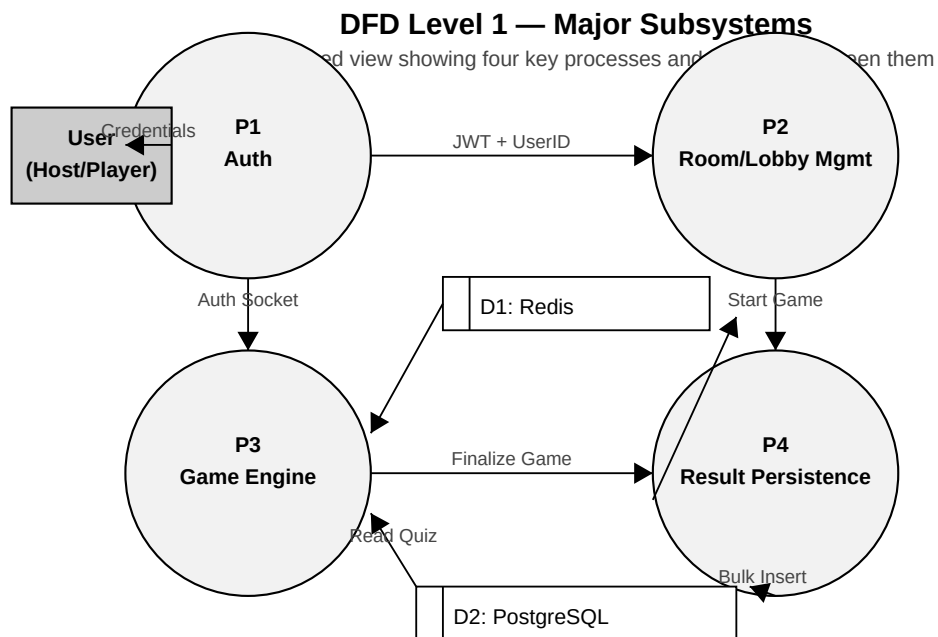


Figure 3.3: DFD Level 1 (Major Subsystems and Inter-Process Data Flow)

The DFD Level 2 zooms into the Game Engine (P3) process, outlining the detailed logic of the websocket event-loop, answer verification, Redis scoring, and final data migration.

DFD Level 2 — Game Engine (P3 Expanded)

Internal processes of the Socket.IO Game Service

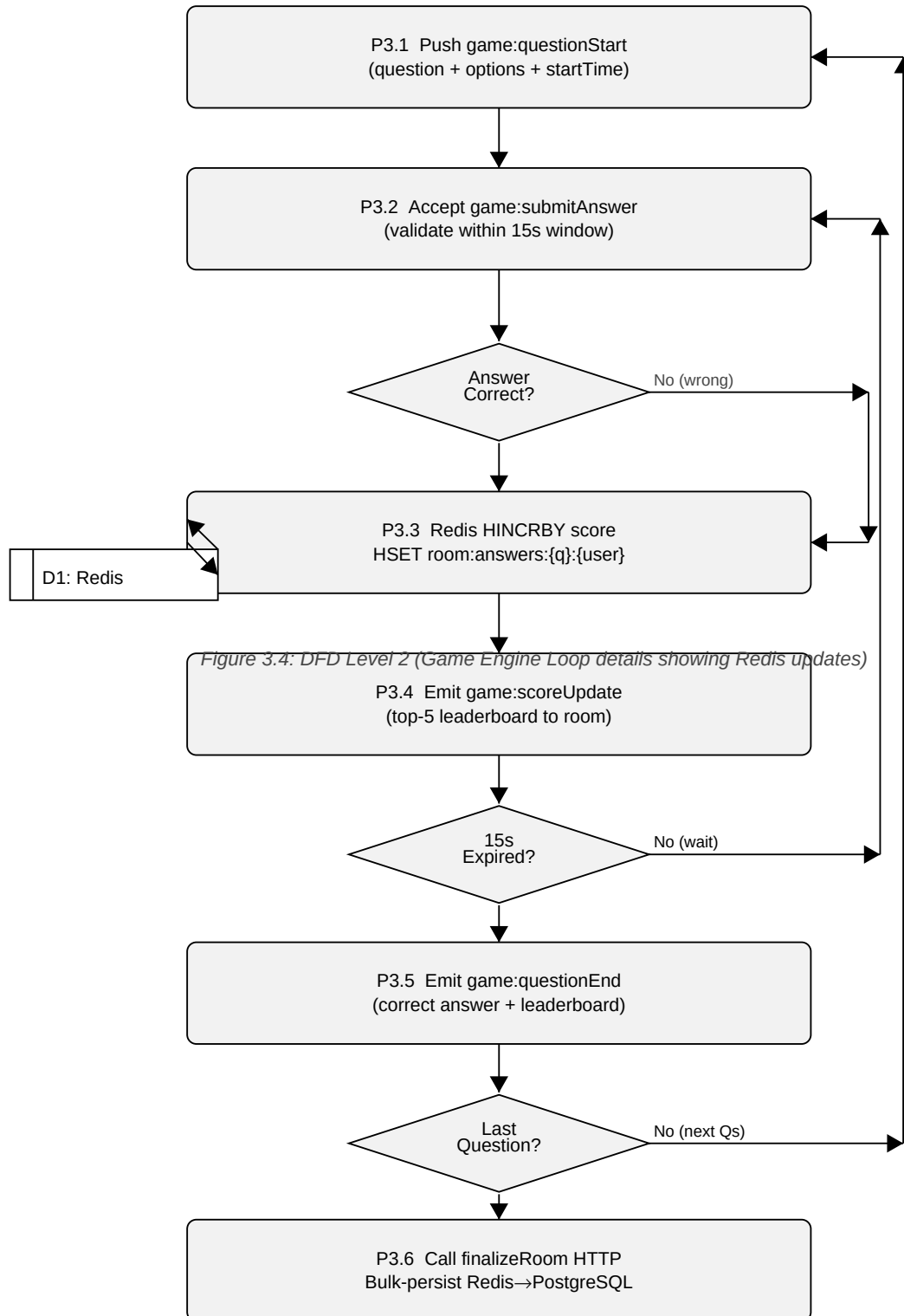


Figure 3.4: DFD Level 2 (Game Engine Loop details showing Redis updates)

3.8 Use Case Diagram / Descriptions

The Use Case Diagram displays relationships between external users (Host and Player) and the operations within the system boundary.

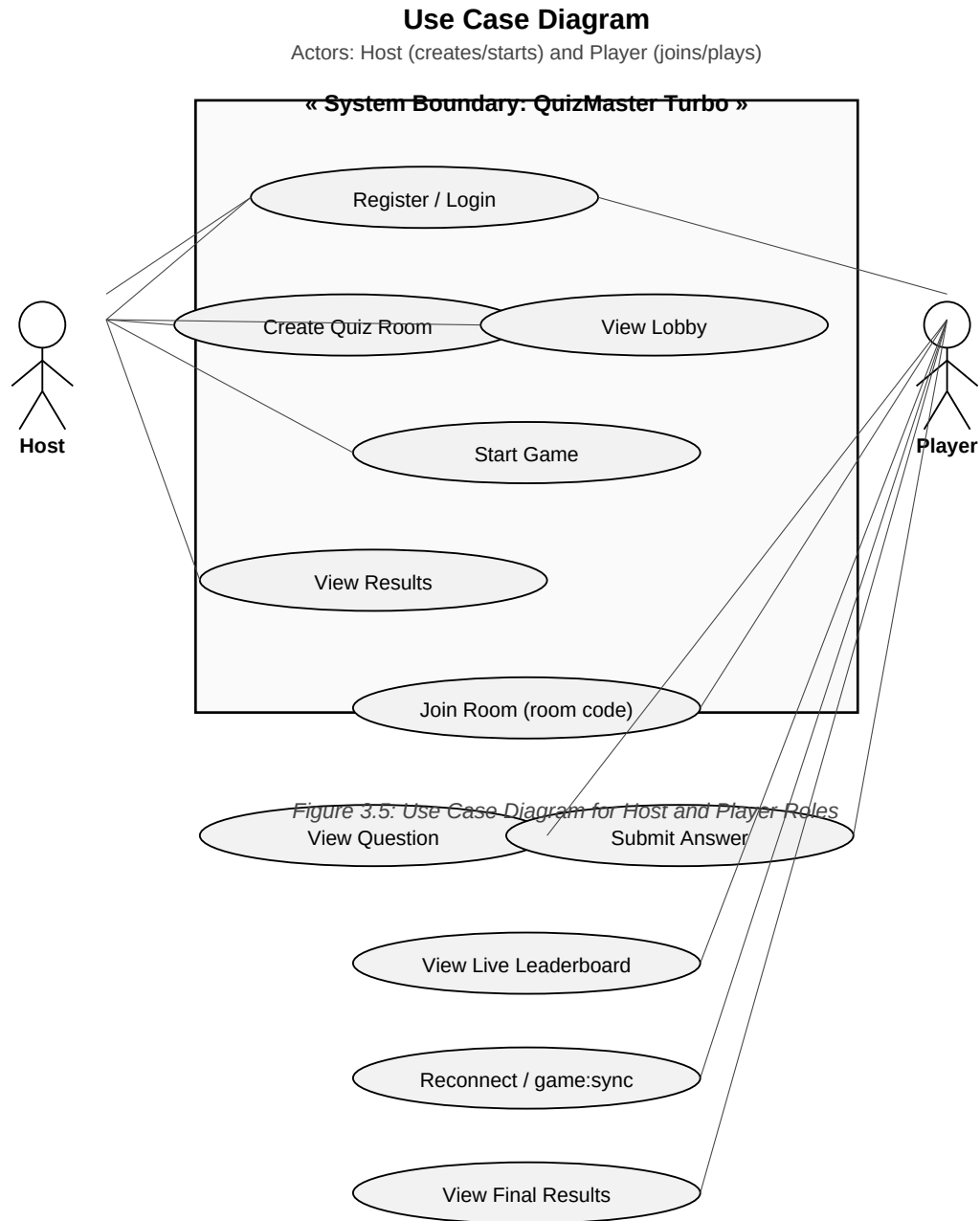


Figure 3.5: Use Case Diagram for Host and Player Roles

3.9 Database Design (Class / Entity-Relationship Diagrams)

The Class Diagram details the object-oriented structure of the Prisma models matching the database relational schemas.

Class Diagram — Prisma Schema Models

All 10 models from the actual schema.prisma file



The Entity-Relationship Diagram (ERD) defines tables, fields, constraints, primary keys (PK), foreign keys (FK), and cardinalities (1-to-N relationships).

Entity-Relationship Diagram

Based on actual Prisma schema — Room is aggregate root for a game session

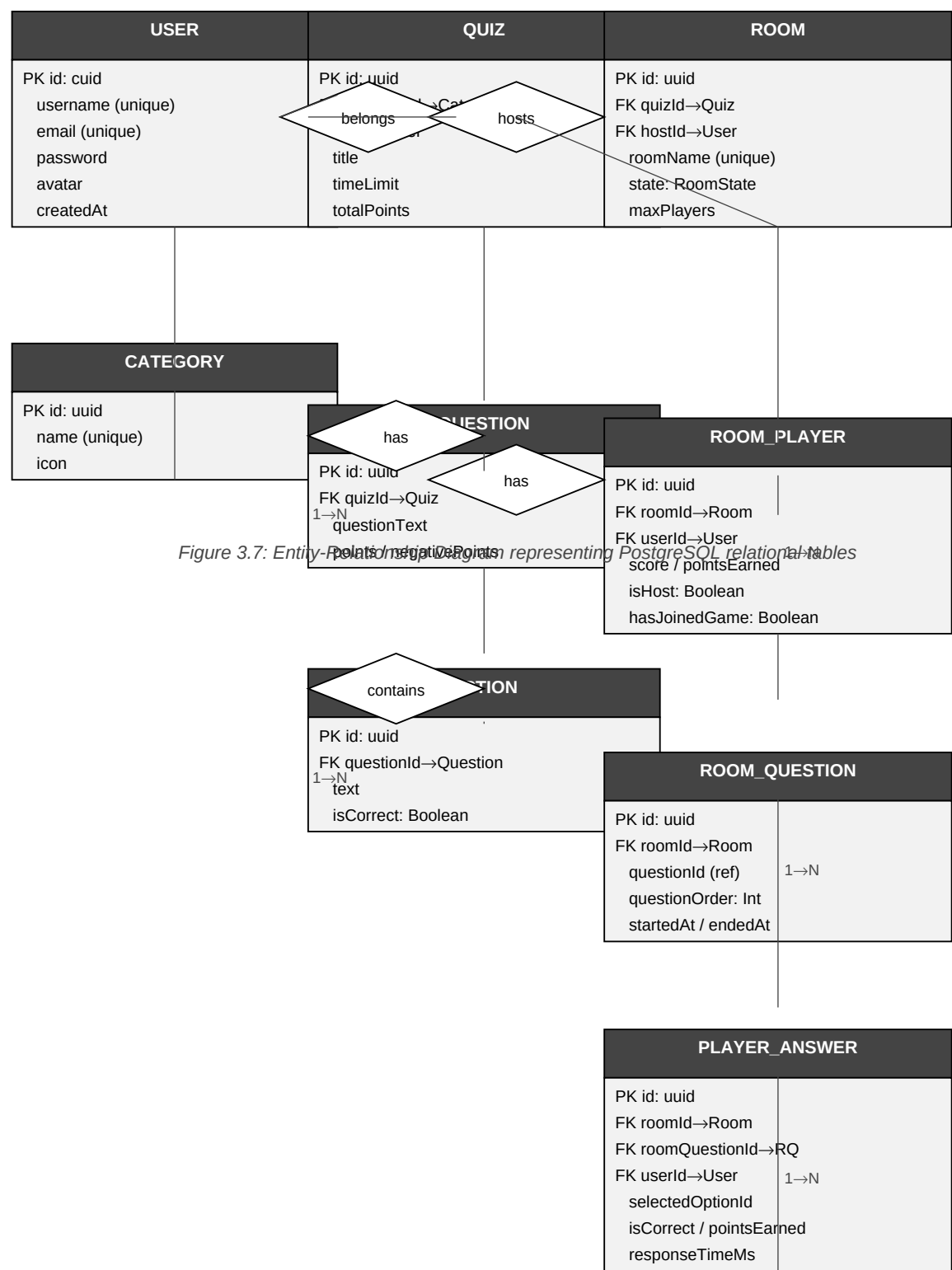


Figure 3.7: Entity-Relationship Diagram representing PostgreSQL relational tables

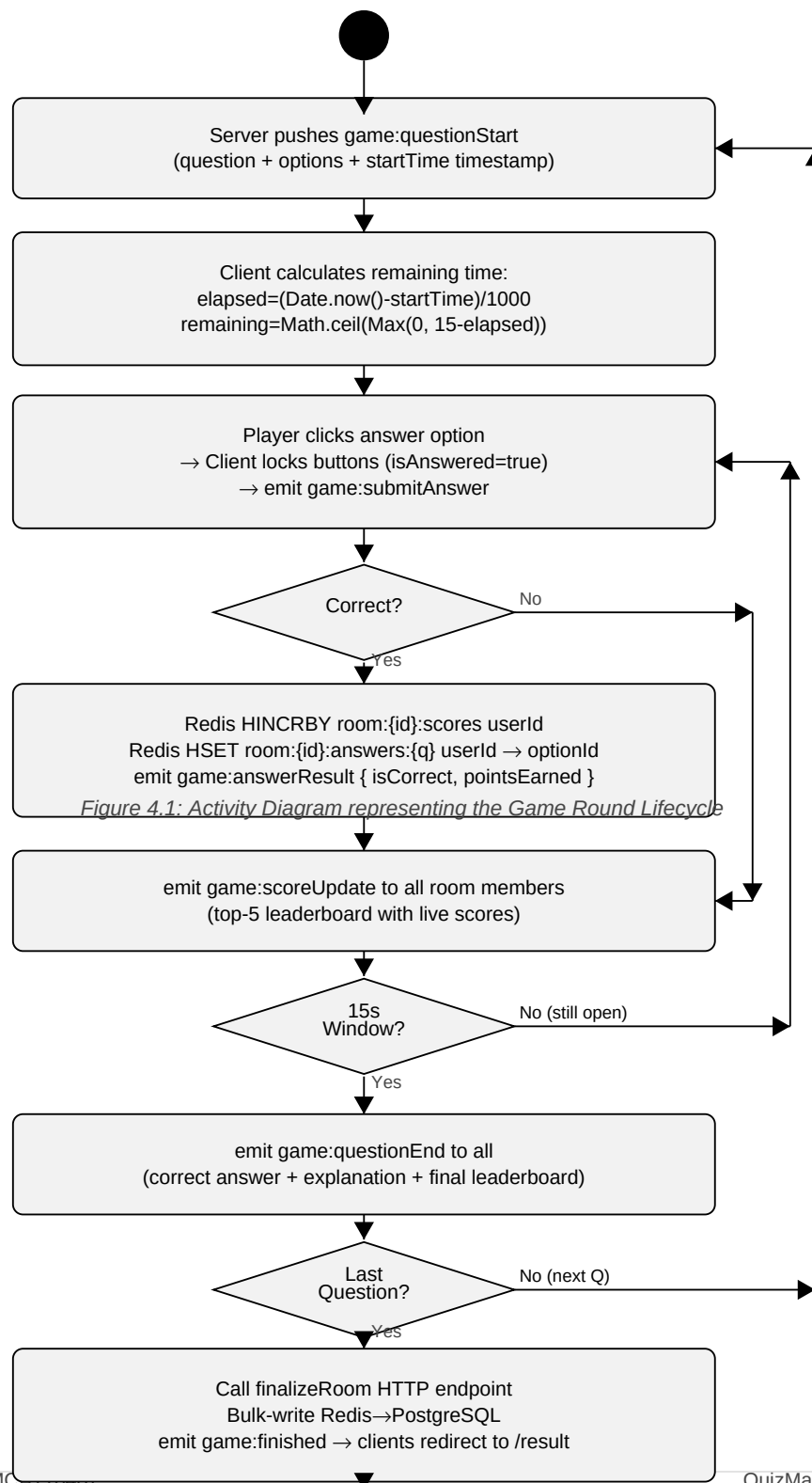
Chapter 4: System Design

4.1 Activity Diagram

The Activity Diagram maps the dynamic, event-driven lifecycle of a quiz round, modeling actions from the initial question push to timer expiration and next question transitions.

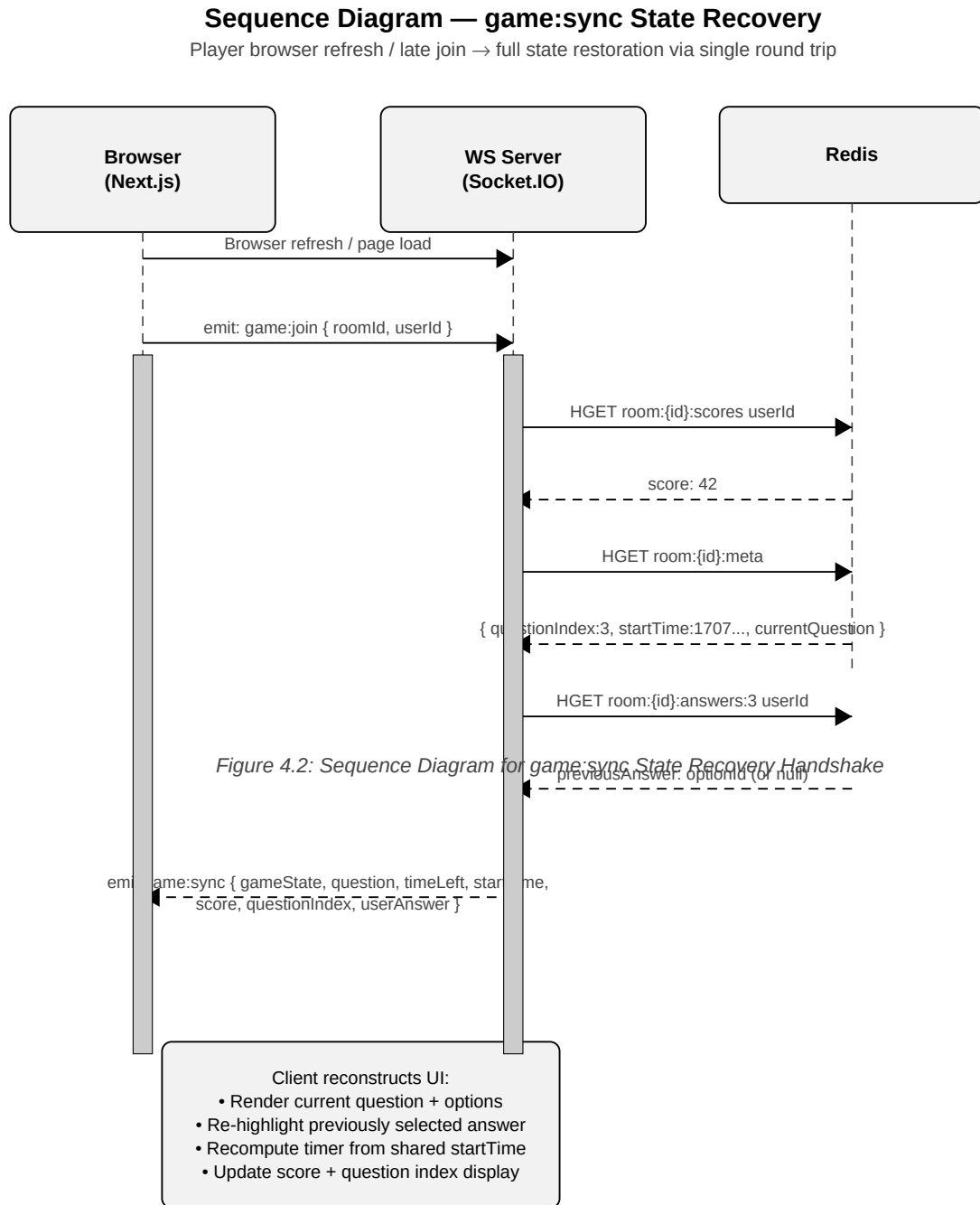
Activity Diagram — Game Round Lifecycle

Complete flow from question push to next question or finalization



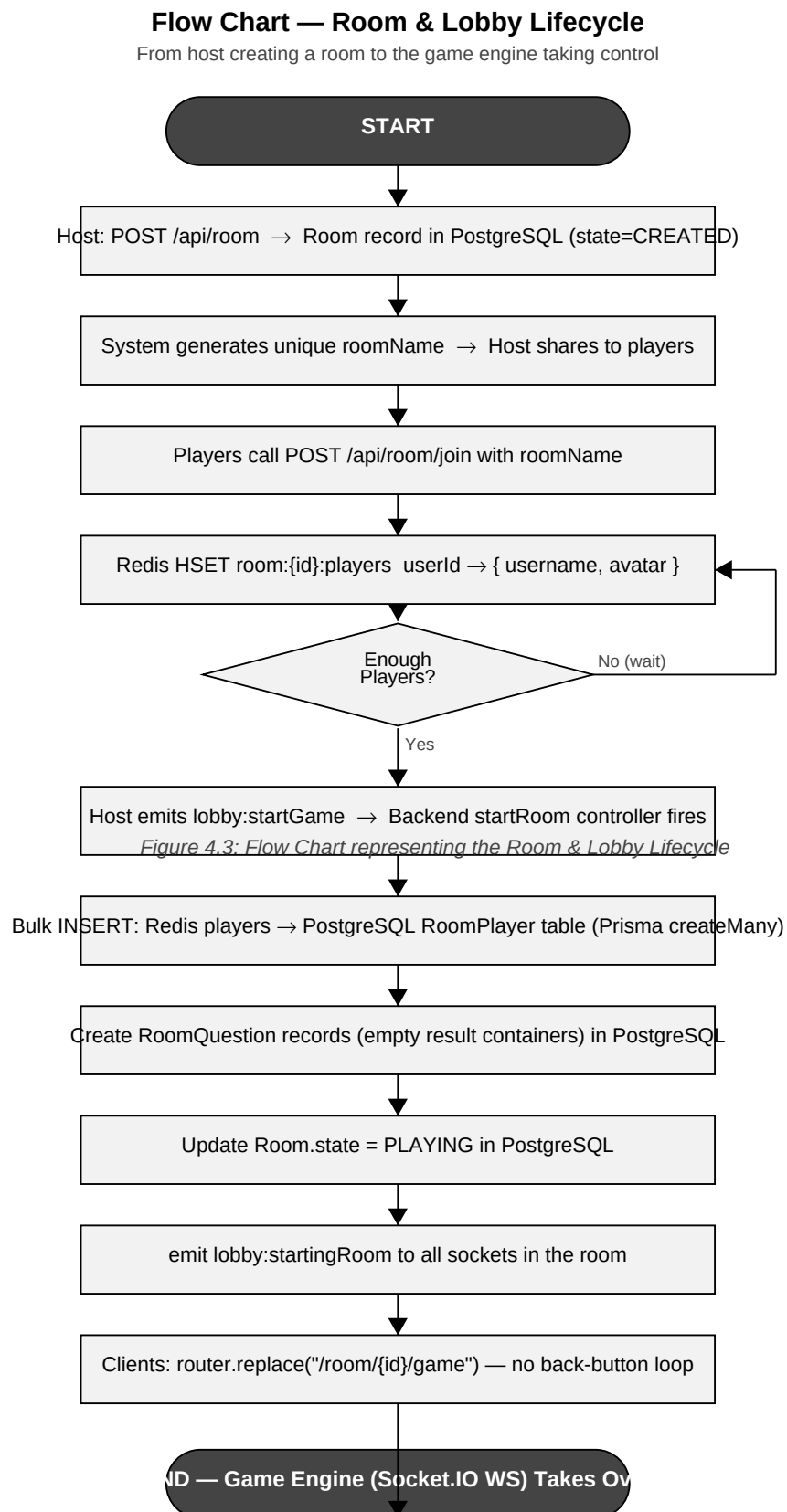
4.2 Sequence Diagram

The Sequence Diagram details the exact timeline of the state recovery (game:sync) handshake, showing the messages passed between Browser, WS Server, and Redis cache.



4.3 Flowchart

The Flowchart outlines the logical steps involved in room creation, player joining, and transition into active gameplay.



4.4 Detailed Schema Specification

Below is the relational mapping of the primary models defined in the database schema:

Table Name	Primary Key (PK)	Foreign Keys (FK)	Key Attributes
User	id (cuid)	None	username, email, password, avatar, createdAt
Quiz	id (uuid)	categoryId	quizNumber, title, timeLimit, totalPoints
Question	id (uuid)	quizId	questionText, points, negativePoints
Room	id (uuid)	quizId, hostId	roomName, state, maxPlayers, startedAt
RoomPlayer	id (uuid)	roomId, userId	score, pointsEarned, hasJoinedGame

Sample Client Screenshot Placeholder

The interface uses a dark-themed responsive glassmorphic card design displaying the question prompt, answer choice buttons, a countdown wheel, and live rankings.

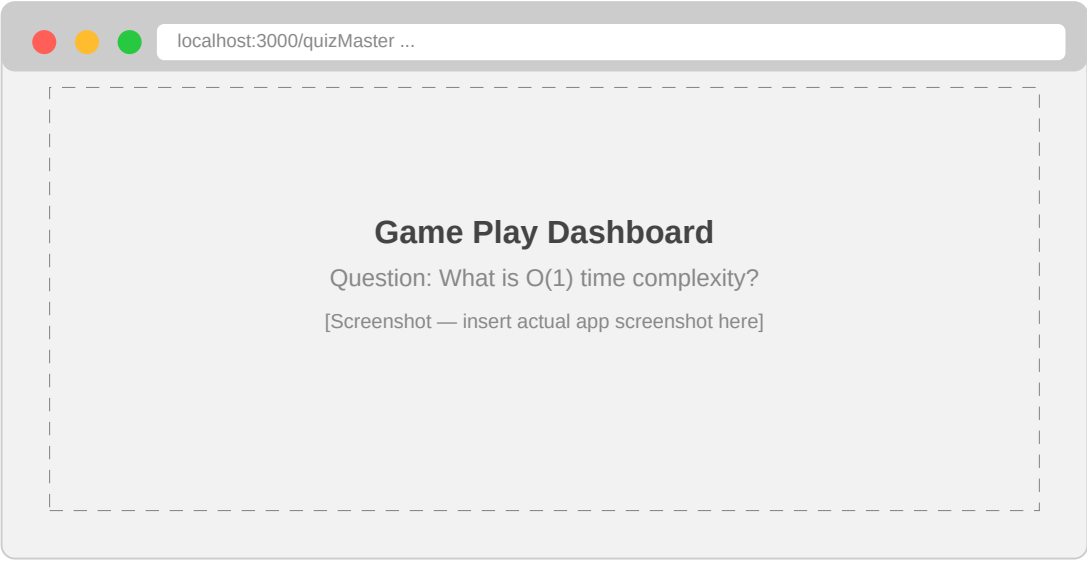


Figure 4.4: Next.js Client Dashboard Screenshot Placeholder

Chapter 5: Implementation & Testing

5.1 System Specifications (Hardware & Software)

The platform was compiled and executed under the following technical conditions:

Resource Type	Development Specification	Deployment Target
Processor (CPU)	Intel Core i5-11400H @ 2.70GHz (6 Cores)	Shared VPS Intel Xeon (2 vCPUs)
Memory (RAM)	16.0 GB DDR4 System RAM	2.0 GB Server RAM
Operating System	Windows 11 Home (x64 Build)	Ubuntu 22.04 LTS (Linux Kernel)
Node.js Environment	Node.js v20.11.0 / npm v10.2.4	Node.js v20 LTS Runtime
Databases	PostgreSQL v16.1 & Redis v7.2 (Docker)	Neon PG Serverless & Upstash Redis

5.2 Implementation Highlights

The core backend game controller coordinates the 15-second round cycles using active timers. The following block highlights the server-side Socket.IO handler:

```
// Server-side Socket.IO Game Controller snippet
socket.on('game:submitAnswer', async ({ roomId, userId, optionId }) => {
  const key = `room:${roomId}:answers:${currentQuestionIndex}`;
  const timeKey = `room:${roomId}:meta`;

  // 1. Fetch start time from Redis to validate answer time limit
  const startTime = await redis.hget(timeKey, 'startTime');
  const elapsed = Date.now() - Number(startTime);
  if (elapsed > 15000) return socket.emit('game:error', 'Time window expired');

  // 2. Write answer to Redis cache (O(1) complexity)
  await redis.hset(key, userId, optionId);

  // 3. Increment score if answer is correct
  const isCorrect = await checkAnswer(optionId);
  if (isCorrect) {
    await redis.hincrby(`room:${roomId}:scores`, userId, 100);
  }
});
```

5.3 Testing Methodologies

A combination of unit testing (using Jest) and integration load testing was performed to validate stability:

Test Case ID	Module Tested	Input / Scenario	Expected Output	Result
TC-AUTH-01	Authentication	Register duplicate username	Return 409 Conflict	Passed
TC-LOBB-03	Lobby Manager	Client joins invalid room code	Emit room:notFound socket error	Passed
TC-GAME-02	Game Engine	Submit answer after 16 seconds	Ignore submission, score = +0	Passed
TC-SYNC-01	Reconnection	Refresh page during Q3 at 6s elapsed	UI reloads Q3 showing 9s left	Passed
TC-PERS-04	Persistence	Complete last question of room	Prisma writes stats, Redis keys deleted	Passed

Chapter 6: Conclusion & Future Scope

6.1 Summary of Findings

QuizMaster Turbo successfully demonstrates the viability of a hybrid persistence model for real-time web gaming. Using Socket.IO web sockets allows instantaneous event delivery, while Redis bypasses traditional relational database bottlenecks by keeping ephemeral connection states and live scoreboards in memory. The Next.js client renders dynamic screens responsively, and the state-recovery protocol effectively handles unexpected network disconnections, achieving a premium, robust user experience.

6.2 Limitations of the System

Despite meeting its core goals, the prototype contains certain limitations:

- **Redis Memory Bounds:** Since Redis holds active states in RAM, a huge spike in concurrent rooms could lead to server memory exhaustion if expired keys are not actively cleaned up.
- **Single Server Limitation:** Without a Redis Pub/Sub adapter, socket rooms must reside on the same Express instance. Scaling across multiple container instances requires configuring a Socket.IO Redis Adapter.
- **Static Question Formats:** Currently, the platform supports multiple-choice questions only, lacking text inputs, match-the-following, or audio/video attachments.

6.3 Future Enhancements

Planned improvements for future iterations include:

- **Horizontal Clustering:** Deploying Socket.IO Redis Adapters to cluster socket servers across multi-instance Kubernetes pods.
- **Rich Media Support:** Allowing creators to attach image, audio, and YouTube video embeds to questions.
- **Advanced Anti-Cheat:** Tracking tab changes or browser focus loss to flag cheating attempts by participants.

Bibliography / References

[1] R. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[2] K. Chodorow, MongoDB: The Definitive Guide, 2nd ed. O'Reilly Media, 2013.

[3] B. Segers, "Real-time web communication using WebSockets," International Journal of Web Engineering, vol. 12, no. 4, pp. 112–119, 2021.

[4] Socket.IO Documentation, "Connection State Recovery," [Online]. Available: <https://socket.io/docs/v4/connection-state-recovery/>. Accessed June 2026.

[5] Prisma ORM Reference, "Database connections and migrations with PostgreSQL," [Online]. Available: <https://www.prisma.io/docs/concepts/database-connection/>. Accessed June 2026.

[6] Redis Inc., "Redis Commands and Data Structures Reference," [Online]. Available: <https://redis.io/commands/>. Accessed June 2026.

[7] M. Haverbeke, Eloquent JavaScript: A Modern Introduction to Programming, 3rd ed. No Starch Press, 2018.